Q5: Python bez rozbudowanego kodowania

Oba zadania krok po kroku, możliwe odpowiedzi i ocena doświadczenia

Data Platform Engineer / AWS DevOps · przewodnik po polsku · 17 października 2026

Materiał dla prowadzącego, który chce rozumieć kod przed zadaniem pytania. Obejmuje dwa krótkie zadania i rozmowę o rzeczywistej automatyzacji. Wyjaśnienia są po polsku, pytania do kandydata po angielsku.

Q5 trwa 10 minut: 4 minuty na dwa błędy pierwszej funkcji, 2 na warunek logiczny w drugiej i 4 na doświadczenie. Kandydat może odpowiadać ustnie: bez edytora, implementowania rozbudowanego rozwiązania, boto3 i pisania testów.

Najważniejsze: oba błędy pierwszego zadania są wymagane. Drugi z nich wyraźnie pokazuje się po naprawieniu pierwszego. Rekordy to przykładowe dane ćwiczenia, nie dosłowna odpowiedź API AWS.

Spis treści

1. Cel pytania i poziom trudności 2
2. Słownik Pythona potrzebny prowadzącemu 2
3. Zadanie 1: dokładny kod i pytanie 3
4. Pierwszy błąd: return jest wewnątrz pętli 3
5. Drugi błąd: domyślna lista żyje między wywołaniami 4
6. Pełna poprawa i dodatkowa obserwacja o referencjach 4
7. Przypadki brzegowe i podpowiedzi w zadaniu 1 5
8. Zadanie 2: dokładny kod i oczekiwane wyniki 6
9. Dlaczego warunek z or jest zawsze prawdziwy 6
10. Mała poprawa i co wystarczy sprawdzić 7
11. Rozmowa o własnej automatyzacji: pozostałe 4 minuty 7
12. Timeout, retry i testy bez nadmiernej implementacji 8
13. Możliwe odpowiedzi i jak je interpretować 8
14. Ocena i kontrola czasu 9
15. Ściąga przed zadaniem pytania 10

1. Cel pytania i poziom trudności

Sprawdzasz czytanie prostego kodu używanego w automatyzacji: iterację po zadaniach, wybór błędnych uruchomień i niezależność kolejnych wywołań. Nie sprawdzasz algorytmów konkursowych ani szybkości pisania.

Część	Co kandydat powinien wyjaśnić
Zadanie 1	Przedwczesny return i współdzielona lista domyślna
Zadanie 2	Dlaczego niepusty napis przepuszcza wszystkie rekordy
Doświadczenie	Własna automatyzacja, błąd, sposób wykrycia i sprawdzenie naprawy

Dla regular/mid AWS DevOps to podstawy przydatne w skryptach operacyjnych. Nie zakładaj jednak, że jeden krótki błąd rozstrzyga całą ocenę Pythona. Zapisz samodzielność, użyte podpowiedzi i dowód z praktyki.

Wprowadzenie po angielsku:

> “This is a short verbal code review. You do not need to write a full solution. Please explain what the code does and the small changes you would make.”

2. Słownik Pythona potrzebny prowadzącemu

Zapis	Znaczenie w tych zadaniach
`def`	Definicja funkcji, czyli fragmentu kodu wywoływanego po nazwie
`jobs`	Parametr: lista rekordów przekazana funkcji
`[]`	Nowa pusta lista
`{"name": ..., "status": ...}`	Słownik: rekord z wartościami pod wskazanymi kluczami
`job["status"]`	Pobranie statusu pojedynczego rekordu
`for job in jobs`	Przejście po kolejnych elementach listy
`==`	Porównanie wartości, nie przypisanie
`if`	Wykonanie zagnieżdżonego bloku, jeśli warunek jest prawdziwy
`.append(...)`	Dopisanie elementu do istniejącej listy
`return`	Natychmiastowe zakończenie funkcji z wynikiem

Wcięcia określają, do którego bloku należy instrukcja. To nie jest wyłącznie formatowanie. `return` w pętli kończy funkcję już podczas pierwszego przejścia, nawet jeśli pętla miała obejrzeć sto rekordów.

Nie myl `return` z wypisaniem wyniku. Funkcja może zwrócić listę bez pokazania jej użytkownikowi. Jeśli dochodzi do końca bez wykonania return, zwraca `None`, czyli specjalną wartość oznaczającą brak takiego wyniku. Nie jest to pusta lista.

3. Zadanie 1: dokładny kod i pytanie

```
jobs = [
    {"name": "load_customers", "status": "SUCCEEDED"},
    {"name": "load_orders", "status": "FAILED"},
    {"name": "load_products", "status": "FAILED"},
]

def failed_jobs(jobs, failed=[]):
    for job in jobs:
        if job["status"] == "FAILED":
            failed.append(job["name"])
        return failed
```

Cel: każdorazowo zwrócić nazwy **wszystkich** nieudanych zadań z otrzymanej listy, bez pamiętania wyników z poprzednich wywołań.

> “This function should return all failed job names. What does it actually return for the sample input? What change lets it examine every job? After making only that change, what do two successive calls to failed_jobs(jobs) return? Assume a fresh function definition and no other calls. Explain the second bug and its correction. After both fixes, what should happen for an empty list or when no jobs failed?”

Możesz podawać te części kolejno. „Fresh function definition” oznacza, że zaczynamy z nowo utworzoną listą domyślną, a nie ze stanem zostawionym przez wcześniejsze próby. Bez tego założenia nie byłoby jednego pewnego wyniku.

4. Pierwszy błąd: return jest wewnątrz pętli

Prześledź oryginalną funkcję z nową, jeszcze pustą listą domyślną:

Krok	Co się dzieje
1	`job` oznacza pierwszy rekord: load_customers
2	Status to SUCCEEDED, więc warunek FAILED jest fałszywy
3	append nie wykonuje się; lista failed nadal jest pusta
4	return wykonuje się niezależnie od if, ale wewnątrz for
5	Funkcja kończy się z wynikiem `[]`
6	load_orders i load_products nie zostają sprawdzone

Poprawa pierwszego błędu: przenieść `return failed` poza pętlę, na poziom wcięcia instrukcji `for`. Nie trzeba przesuwać go do if ani usuwać całkowicie.

> “Which record is processed immediately before the function returns?”

To dobre dopytanie, jeśli kandydat mówi ogólnie „problem ze wcięciami”. Powinien połączyć konkretne wcięcie z konkretnym wynikiem.

Pułapka dla prowadzącego: oryginalny kod przy pustej liście jobs w ogóle nie wchodzi do pętli, więc nie wykonuje return i zwraca `None`. Po pełnej naprawie oczekujemy `[]`.

5. Drugi błąd: domyślna lista żyje między wywołaniami

Domyślna wartość parametru powstaje, gdy wykonywana jest definicja funkcji. Nie jest tworzona od nowa przy każdym wywołaniu. Gdy jest listą, append zmienia ten sam obiekt używany przez kolejne wywołania bez drugiego argumentu. Python: współdzielone wartości domyślne.

Analogia: funkcja dostaje na stałe jeden notes. Każde wywołanie dopisuje do tego samego notesu, zamiast otwierać nową pustą kartkę.

Ważna kolejność w rozmowie: najpierw naprawiamy tylko wcięcie return, pozostawiając `failed=[]` w parametrach. Rozpatrujemy świeżą definicję:

```
def failed_jobs(jobs, failed=[]):
    for job in jobs:
        if job["status"] == "FAILED":
            failed.append(job["name"])
    return failed
```

Kolejne wywołanie tej samej definicji	Wynik widoczny bezpośrednio po wywołaniu
Pierwsze `failed_jobs(jobs)`	`["load_orders", "load_products"]`
Drugie `failed_jobs(jobs)`	`["load_orders", "load_products", "load_orders", "load_products"]`
Następne `failed_jobs([])`	Ta sama lista czterech wcześniejszych nazw

Ostatni wynik pokazuje sedno: funkcja zwraca stare błędy, choć nowe wejście nie zawiera żadnego zadania. Tak samo może się zachować wejście z samymi sukcesami. Wynik zależy od historii wywołań, a powinien tylko od aktualnego wejścia.

W oryginalnym przykładzie pierwszy rekord jest sukcesem, więc przedwczesny return **ukrywa dopisywanie**. Nie oczekuj narastania wyniku tej konkretnej oryginalnej próbki przed poprawieniem wcięcia.

6. Pełna poprawa i dodatkowa obserwacja o referencjach

Najprostszy czytelny wariant referencyjny dla prowadzącego:

```
def failed_jobs(jobs):
    failed = []
    for job in jobs:
        if job["status"] == "FAILED":
            failed.append(job["name"])
    return failed
```

Lista powstaje **wewnątrz wywołania**. Dwa wywołania dla tego samego wejścia dają takie same dwie nazwy, ale niezależne listy. Puste wejście i same sukcesy dają `[]`, również po wcześniejszych wywołaniach z błędami.

Akceptuj też parametr z domyślnym `None` i utworzenie nowej listy w środku, albo poprawne list comprehension. Nie wymagaj krótszej lub bardziej efektownej składni. Jeśli kandydat zachowuje możliwość przekazania akumulatora, upewnij się, że rozumie wymaganie niezależności standardowych wywołań.

Dodatkowa głębia, nie warunek zaliczenia: po samej poprawie return oba wywołania zwracają referencję do tego samego obiektu. Jeśli zapiszesz pierwszy wynik jako a, a potem wykonasz drugie wywołanie jako b, lista a też będzie już zawierała cztery nazwy. Pierwszy wynik nie jest zamrożoną kopią.

> “Would a saved first result change after the second call? Why?”

Nie rozwijaj tego w zadanie o pamięci, jeśli czas się kończy. Podstawą są dwa błędy i niezależny wynik po poprawie.

7. Przypadki brzegowe i podpowiedzi w zadaniu 1

Poniżej każda pozycja dla oryginalnego kodu zakłada **oddzielną świeżą definicję**, więc wyniki nie kumulują się pomiędzy wierszami tabeli.

Wejście	Oryginalny kod	Po obu poprawkach
Próbka z zadania	`[]`	orders i products
Pusta lista	None	`[]`
Jeden FAILED	Jego nazwa	Jego nazwa
Pierwszy FAILED, drugi FAILED	Tylko pierwsza nazwa	Obie nazwy
Same SUCCEEDED	`[]`	`[]`

Zwróć uwagę: jeden FAILED może „przejść test”, mimo że funkcja ma oba błędy. Dlatego potrzeba listy z kilkoma rekordami i wielokrotnych wywołań.

Stopniowanie podpowiedzi, jeśli są potrzebne:

1. “At which point does return end the function?”
2. “Now imagine return is outside the loop. What happens on the second call?”
3. “When is the default list created: at definition time or on every call?”

Trzecie pytanie podaje już znaczną część wyjaśnienia. Kandydat, który dopiero wtedy dochodzi do rozwiązania, wykazał mniej samodzielności niż osoba, która od początku przewidziała narastający wynik. Zanotuj różnicę bez automatycznej dyskwalifikacji i bez udawania, że odpowiedzi były jednakowo samodzielne.

8. Zadanie 2: dokładny kod i oczekiwane wyniki

```
runs = [
    {"name": "load_customers", "status": "SUCCEEDED"},
    {"name": "load_orders", "status": "FAILED"},
    {"name": "load_products", "status": "TIMED_OUT"},
    {"name": "load_prices", "status": "CANCELLED"},
]

def retryable_jobs(jobs):
    retryable = []
    for job in jobs:
        if job["status"] == "FAILED" or "TIMED_OUT":
            retryable.append(job["name"])
    return retryable
```

> “What does retryable_jobs(runs) return, and what should it return? Why does this condition also select successful or cancelled jobs? Explain a small correction and which inputs you would check. No implementation is required.”

Funkcja powinna wybrać FAILED i TIMED_OUT. Błędny kod wybiera **wszystkie cztery nazwy**, w kolejności wejścia. Poprawny wynik to tylko `load_orders` i `load_products`.

W tym zadaniu lista wynikowa już powstaje w środku, a return jest poza pętlą. Nie przenoś mechanicznie diagnozy z pierwszego ćwiczenia. Tutaj błąd jest w warunku logicznym.

9. Dlaczego warunek z or jest zawsze prawdziwy

Program nie odczytuje tego jak polskiego zdania „status jest FAILED lub TIMED_OUT”. Widzi dwie niezależne części: porównanie statusu z FAILED oraz sam napis TIMED_OUT.

Niepusty napis jest traktowany jako prawdziwy w warunku. `or` nie dopisuje automatycznie brakującego porównania. W rezultacie nawet gdy lewa część jest fałszywa, prawa jest prawdziwa. Python: wartości logiczne.

Status	`status == "FAILED"`	Prawa część: niepusty napis	Warunek w if
SUCCEEDED	Fałsz	Prawdziwa	Prawdziwy
FAILED	Prawda	Nie trzeba jej obliczać	Prawdziwy
TIMED_OUT	Fałsz	Prawdziwa	Prawdziwy
CANCELLED	Fałsz	Prawdziwa	Prawdziwy

> “For SUCCEEDED, what is the left side of or, and what is the right side?”

To lepsza próba zrozumienia niż pytanie o sam termin „truthy”. Kandydat może nie użyć tego słowa i nadal poprawnie wyjaśnić problem.

Opcjonalna głębia: Pythonowy `or` zwraca jeden z operandów i działa z krótkim spięciem, a nie zawsze zwraca dosłownie `True` lub `False`. W if wynik jest oceniany logicznie. Ta dokładność jest plusem, nie wymogiem do rozwiązania dwuminutowego zadania.

10. Mała poprawa i co wystarczy sprawdzić

Czytelny wariant referencyjny:

```
def retryable_jobs(jobs):
    retryable = []
    for job in jobs:
        if job["status"] in ("FAILED", "TIMED_OUT"):
            retryable.append(job["name"])
    return retryable
```

`in` pyta, czy status należy do wskazanego zbioru dozwolonych wartości. Akceptuj również dwa pełne porównania połączone or:

```
if job["status"] == "FAILED" or job["status"] == "TIMED_OUT":
    retryable.append(job["name"])
```

Samo zastąpienie or przez and nie naprawia wymagania. Lewa część nadal sprawdza tylko FAILED, a TIMED_OUT nie jest porównany ze statusem. Dodanie nawiasów wokół pierwotnego warunku również nie wystarcza.

Wystarczy ustnie wymienić sprawdzenia: mieszana próbka, osobno FAILED, TIMED_OUT, SUCCEEDED i CANCELLED oraz pusta lista. Ostatnia zwraca `[]` zarówno w błędnym, jak i poprawnym kodzie, więc sama nie wykrywa tego błędu.

Granica ćwiczenia: zakładamy wymagane klucze name i status. Obsługa brakujących kluczy, normalizacji wielkości liter czy nieznanych statusów to dodatkowa rozmowa o kontrakcie wejścia, nie ukryte obowiązkowe błędy.

11. Rozmowa o własnej automatyzacji: pozostałe 4 minuty

> “Describe one Python automation you personally implemented. What did it do, how did you handle failures, and how did you check it before deployment?”

Może opisać audyt bucketów, pobieranie sekretów, migrację, integrację lub inne zadanie. Nie żądaj implementacji boto3. Poproś o wejście, wykonane operacje, wynik i jedną sytuację błędną.

Dydaktyczny przykład dobrej struktury odpowiedzi: skrypt sprawdza ustawienia bucketów. Wejściem jest zatwierdzona lista kont i bucketów. Wynikiem raport z poprawnymi odczytami, wykrytymi odchyleniami i błędami dostępu. Nieudane odczyty nie są mylone z „wszystko zgodne”. Log zawiera identyfikator wykonania, zasób i typ błędu, ale nie sekret.

To nie jest potwierdzony projekt kandydata. W rozmowie proś o jego rzeczywisty przykład i rozróżniaj pracę własną od rozwiązania całego zespołu.

Wybierz jedno dopytanie:

> “How did you make a partially successful run visible to its caller or operator?”

> “What was one actual bug you found, and what evidence confirmed the fix?”

Dobra odpowiedź może opisać jawny status częściowego błędu, raport nieudanych elementów lub alarm. Sam `try/except` bez informacji o skutku nie wystarcza.

12. Timeout, retry i testy bez nadmiernej implementacji

> “If an API call timed out, how would you decide whether retrying it was safe?”

Timeout mówi, że klient nie uzyskał odpowiedzi w czasie. Nie dowodzi, że usługa nie wykonała operacji. Ponowienie odczytu i ponowne utworzenie zasobu mogą mieć inne skutki. Kandydat powinien uwzględnić możliwość częściowego sukcesu oraz identyfikację operacji lub sprawdzenie jej wyniku.

Przykład: API przyjęło żądanie utworzenia zasobu, ale odpowiedź zaginęła. Ślepe powtórzenie może utworzyć drugi zasób albo zgłosić konflikt. Bezpieczne postępowanie zależy od mechanizmu danej operacji: identyfikatora żądania, idempotentnego zapisu lub sprawdzenia stanu.

> “Which cases did you check, and how did you isolate external services during testing?”

Szukaj przypadków normalnych, pustego wejścia, odmowy dostępu, timeoutu i częściowego błędu. Mock może symulować odpowiedź API i sprawdzić ścieżkę obsługi błędu. Nie dowodzi prawdziwego dostępu w AWS; to wymaga właściwego sprawdzenia integracyjnego.

Nie wymagaj kodu testów, nazw wszystkich wyjątków SDK ani kompletnego mechanizmu retry. Dla tego pytania liczy się zrozumienie skutku ponowienia i potwierdzenie, że automatyzacja nie ukrywa porażki.

13. Możliwe odpowiedzi i jak je interpretować

Odpowiedź kandydata	Co zrobić dalej
“The return is in the wrong place.”	Poproś o aktualny wynik próbki i rekord, na którym kończy
“Never use a mutable default.”	Poproś o dwa konkretne wyniki po samej poprawie return
“The list resets each call.”	Zapytaj, kiedy wykonuje się definicja wartości domyślnej
“All jobs are selected because the string is truthy.”	Poproś o małą poprawę; sens już został uchwycony
“I would change or to and.”	Prześledź TIMED_OUT po tej zmianie
“I would use a comprehension.”	Akceptuj, jeśli wyjaśnia oba błędy, a nie tylko zastępuje kod
“I mainly modified existing scripts.”	Poproś o własną zmianę, błąd i sprawdzenie

Przykładowa dobra odpowiedź do zadań:

> “The first function returns after the first record, so this sample returns an empty list. Moving return outside the loop lets it inspect all records, but the default list is shared. With a fresh definition, the first call returns the two failed names and the second adds the same names again. I would create a new list inside each call. In the second exercise, TIMED_OUT is a non-empty string, not a second comparison, so every record passes. I would use membership in the two allowed statuses.”

Spójna odpowiedź może być znacznie krótsza. Nie wymagaj pamięciowego odtworzenia angielskiego tekstu ani identycznych nazw zmiennych.

14. Ocena i kontrola czasu

Czas	Zadanie
0-4 min	Return, wynik próbki, współdzielona lista, poprawa i puste wejście
4-6 min	Drugie zadanie: rzeczywisty wynik, przyczyna, poprawa
6-10 min	Własna automatyzacja i jeden przypadek błędny

3: samodzielnie wyjaśnia oba błędy pierwszej funkcji i warunek drugiej; proponuje niezależne wyniki i sensowne sprawdzenia. Potrafi opisać rzeczywisty skrypt i reakcję na błąd. **2:** rozwiązuje po wskazówkach, ale wymaga wsparcia przy historii wywołań lub ocenie skutków. **4:** oprócz podstaw uzasadnia bezpieczne ponowienia i częściowy sukces konkretnym przykładem z pracy. **1:** po wyjaśnieniu nadal nie potrafi prześledzić podstawowego przepływu. **N/O:** brak czasu lub dowodu; nie wpisuj zera.

Zapisuj osobno: return, mutable default, warunek or oraz doświadczenie praktyczne. Dzięki temu jedna mocna część nie ukryje niezweryfikowanej innej.

Jeśli kandydat utknie, użyj jednej podpowiedzi i wróć do doświadczenia, zamiast zużyć całe 10 minut na zagadkę. Nie rozszerzaj zakresu o napisanie retry engine, klas, dekoratorów i pełnej integracji AWS.

15. Ściąga przed zadaniem pytania

Zadanie 1, oryginał: świeża definicja + podana próbka = `[]`. Powód: pierwszy rekord jest sukcesem, return kończy funkcję po tym rekordzie. Puste wejście w oryginale = `None`.

Po samej poprawie wcięcia: nowa definicja; pierwsze wywołanie = orders, products; drugie = orders, products, orders, products. To ta sama lista. Następne puste wejście nadal zwraca stare nazwy.

Po obu poprawkach: lista tworzona wewnątrz wywołania; każde wywołanie niezależne; puste wejście lub brak błędów = `[]`.

Zadanie 2: błędny warunek wybiera customers, orders, products i prices. Poprawny wybiera orders i products. Powód: sam napis TIMED_OUT jest prawdziwy w warunku; or nie dopisuje drugiego porównania. Poprawa: in albo dwa porównania.

Najlepsze dopytania:

> “After fixing only return, what happens on the second call?”

> “For SUCCEEDED, what exactly is evaluated on each side of or?”

Z doświadczenia: co skrypt robił, co poszło źle, jak to pokazał i skąd wiadomo, że naprawa działa. Timeout nie jest dowodem niewykonania operacji.

Nie wymagaj: pisania kodu ani testów. Wymagaj wyjaśnienia, a podpowiedzi zapisuj jako część oceny samodzielności.

Materiały powiązane: zadania dla kandydata, plan rozmowy.